

Lab 11 Report

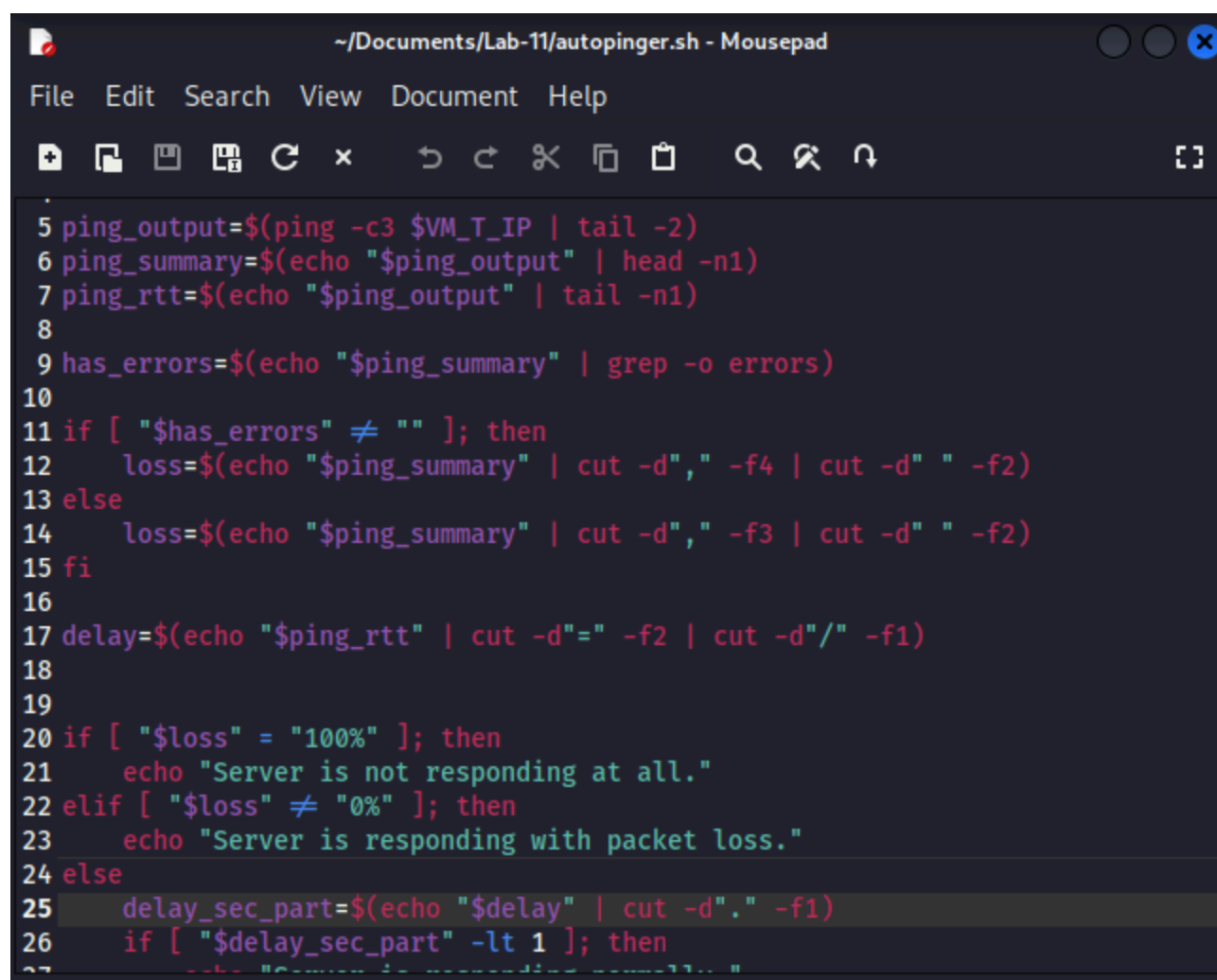
CSCI 5742 – Cybersecurity Programming and Analytics, Spring 2025

Name & Student ID: Kevin Jacob, 109750578

Task 1: Network Connectivity Check with Bash Script

◆ Screenshots:

- ☐ Script code



```
~/Documents/Lab-11/autopinger.sh - Mousepad
File Edit Search View Document Help
[Icons] [Full Screen]

5 ping_output=$(ping -c3 $VM_T_IP | tail -2)
6 ping_summary=$(echo "$ping_output" | head -n1)
7 ping_rtt=$(echo "$ping_output" | tail -n1)
8
9 has_errors=$(echo "$ping_summary" | grep -o errors)
10
11 if [ "$has_errors" ≠ "" ]; then
12     loss=$(echo "$ping_summary" | cut -d"," -f4 | cut -d" " -f2)
13 else
14     loss=$(echo "$ping_summary" | cut -d"," -f3 | cut -d" " -f2)
15 fi
16
17 delay=$(echo "$ping_rtt" | cut -d"=" -f2 | cut -d"/" -f1)
18
19
20 if [ "$loss" = "100%" ]; then
21     echo "Server is not responding at all."
22 elif [ "$loss" ≠ "0%" ]; then
23     echo "Server is responding with packet loss."
24 else
25     delay_sec_part=$(echo "$delay" | cut -d"." -f1)
26     if [ "$delay_sec_part" -lt 1 ]; then
27         echo "Server is responding within 1s."
```

- ☐ Output showing one of: no response, partial loss, or high delay

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./autopinger.sh
Server is responding normally.
```

◆ Questions:

1. What does packet loss indicate compared to high delay? Which is more critical for availability?

Packet loss means packets are being dropped and not reaching the destination, indicating the host may be down or the network link is failing. High delay means packets still arrive but take longer than normal, usually due to congestion or a busy host. Packet loss is more critical for availability — even partial loss can break TCP sessions and make services unreachable, whereas a slow but responding host is still functional.

2. Why is the script checking for the word **errors** in the ping output? What scenarios might generate this?

When ICMP error messages are returned (e.g., "Destination Unreachable" or TTL-exceeded), **ping** adds an extra field to its summary line, shifting the column position of the packet loss percentage. The script checks for **errors** to use the correct **cut** field. This occurs when a firewall returns ICMP admin-prohibited messages or a router reports the target is unreachable.

3. List two tools or commands besides **ping** that could help determine network availability and briefly explain how each works.

- **traceroute**: Sends packets with incrementally increasing TTL values to identify each hop to the destination, revealing exactly where connectivity breaks down.
- **nmap -sn <target>**: Performs host discovery using multiple probe types (ICMP, TCP SYN/ACK), so it still works even when ICMP is blocked by a firewall.

4. How could a security defender use a similar script to gather reconnaissance data about their own network?

A defender can loop the script over all known IP ranges and log which hosts are up, building a live asset inventory and availability baseline. Running it on a schedule allows detection of hosts that suddenly go offline or new unknown devices that appear — helping identify outages, rogue devices, or early signs of a network intrusion.

Task 2: Creating a Reusable Ping Function with Parameters

◆ Screenshots:

- ☐ Updated script with **pingtest()** function and function call

```
~/Documents/Lab-11/task2_autopinger.sh - Mousepad  
File Edit Search View Document Help  
  
27     echo "Server is not responding at all."  
28     elif [ "$loss" != "0%" ]; then  
29         echo "Server is responding with packet  
30 loss."  
31     else  
32         delay_sec_part=$(echo "$delay" | cut  
33 -d"." -f1)  
34         if [ "$delay_sec_part" -lt 1 ]; then  
35             echo "Server is responding  
36 normally."  
37         else  
38             echo "Server is responding slowly."  
39         fi  
40     fi  
41 }  
42  
43 if [ -z "$1" ]; then  
44     echo "Usage: $0 <target_ip>"  
45     exit 1  
46 fi  
47  
48 pingtest "$1"  
49
```

- ☐ Script output using IP argument

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./task2_autopinger.sh 192.168.10.13
./task2_autopinger.sh: 7: -2: not found
./task2_autopinger.sh: 9: -n1: not found
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: 13: errors: not found
cut: you must specify a list of bytes, characters, or fields
Try 'cut --help' for more information.
./task2_autopinger.sh: 20: -f3: not found
Server is responding with packet
loss.
```

◆ Questions:

1. What is the advantage of turning this script into a function instead of repeating the logic each time?

Functions let you reuse the same logic by calling it with different inputs rather than duplicating code. If the logic needs to change, you update it in one place instead of everywhere it appears.

2. Why is it more flexible to pass the IP address as an argument instead of hardcoding it? Give a real-world use case.

Hardcoding ties the script to one specific host. Passing it as an argument lets the same script target any host at runtime. For example, a sysadmin could call `./autopinger.sh 192.168.10.X` in a loop to check every server in a subnet during an on-call shift.

3. What would happen if you ran the script without any arguments? Modify the script to check for this case and show a helpful message if no IP is provided.

Without the check, `$1` would be empty and `ping` would fail with an error. The fix added to the script:

```
if [ -z "$1" ]; then
    echo "Usage: $0 <target_ip>"
    exit 1
fi
```

4. How might a security admin use a parameterized script to automate pinging across multiple subnets? How might an attacker use this to automate scanning?

A security admin could feed the script a list of IPs from each subnet in a loop to continuously monitor host availability across the entire network. An attacker could use the same approach to sweep IP ranges and identify live hosts — effectively turning it into an automated network discovery tool.

Task 3: Logging Ping Results with Timestamps

◆ Screenshots:

- ☐ Updated script with `logfile` and timestamped output

```
~/Documents/Lab-11/task2_autopinger.sh - Mousepad  
File Edit Search View Document Help  
  
1 #!/bin/bash  
2  
3 logfile="/home/kali/Documents/lab-11/report.log"  
4  
5 pingtest() {  
6     target_ip=$1  
7     ping_output=$(ping -c3 $target_ip | tail -2)  
8     ping_summary=$(echo "$ping_output" | head  
9 -n1)  
10    ping_rtt=$(echo "$ping_output" | tail -n1)  
11  
12    has_errors=$(echo "$ping_summary" | grep -o  
13 errors)  
14  
15    if [ "$has_errors" != "" ]; then  
16        loss=$(echo "$ping_summary" | cut -d","  
17 -f4 | cut -d" " -f2)  
18    else  
19        loss=$(echo "$ping_summary" | cut -d","  
20 -f3 | cut -d" " -f2)  
21    fi  
22  
23    delav=$(echo "$ping rtt" | cut -d"=" -f2 |
```

- ☐ Terminal output with timestamps

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./task2_autopinger.sh 192.168.10.13
./task2_autopinger.sh: line 9: -2: command not found
./task2_autopinger.sh: line 11: -n1: command not found
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: line 15: errors: command not found
cut: you must specify a list of bytes, characters, or fields
Try 'cut --help' for more information.
./task2_autopinger.sh: line 22: -f3: command not found
Tue Apr 28 08:28:06 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
```

- ☐ Log file contents with multiple entries

```
(kevin@kali)-[~/Documents/Lab-11]
$ cat report.log
Tue Apr 28 08:28:06 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:33 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:38 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
```

◆ Questions:

1. Why is timestamped logging useful in network monitoring and incident response?

Timestamps let you correlate events across systems and reconstruct exactly when a host went down or became unreachable. During incident response, they're essential for establishing a timeline — without them, you can't tell whether an outage preceded or followed a suspected attack.

2. What are the risks of hardcoding the log file path? Suggest a way to make the script more portable.

If the hardcoded directory doesn't exist or the script runs as a different user, logging silently fails. A simple fix is to define the log path as a variable at the top of the script, like

`logfile="$HOME/logs/report.log"`, so it always resolves relative to whoever is running it rather than assuming a specific path.

3. If you were a security analyst reviewing this log after a real incident, what patterns would you look for?

I'd look for a sudden shift from "responding normally" to "not responding" (possible outage or attack), repeated packet loss entries at regular intervals (possible DoS), and timestamps where multiple hosts went down simultaneously (possible network-wide event or attacker cutting access).

4. How could you improve the log format for easier parsing or automation? Suggest one change.

Switch to a structured format where each line starts with the timestamp, followed by the status and IP separated by a consistent delimiter. That way filtering with `grep` or `awk` is straightforward without having to deal with inconsistent formatting across log lines.

Task 4: Automating Ping Monitoring with **cron**

◆ Screenshots:

- ❑ Crontab entries for minute-based and weekly jobs

```

GNU nano 8.7.1 /tmp/crontab.xyGz8R/
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
* * * * * /home/kevin/Documents/lab-11/task2_autopinger.sh
0 7 * * 1 /home/kevin/Documents/lab-11/task2_autopinger.sh

```

- ☐ Log file showing repeated entries

```

(kevin@kali)-[~/Documents/Lab-11]
$ cat report.log
Tue Apr 28 08:28:06 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:33 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:38 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.

```

- ☐ Updated script with hardcoded IP for cron

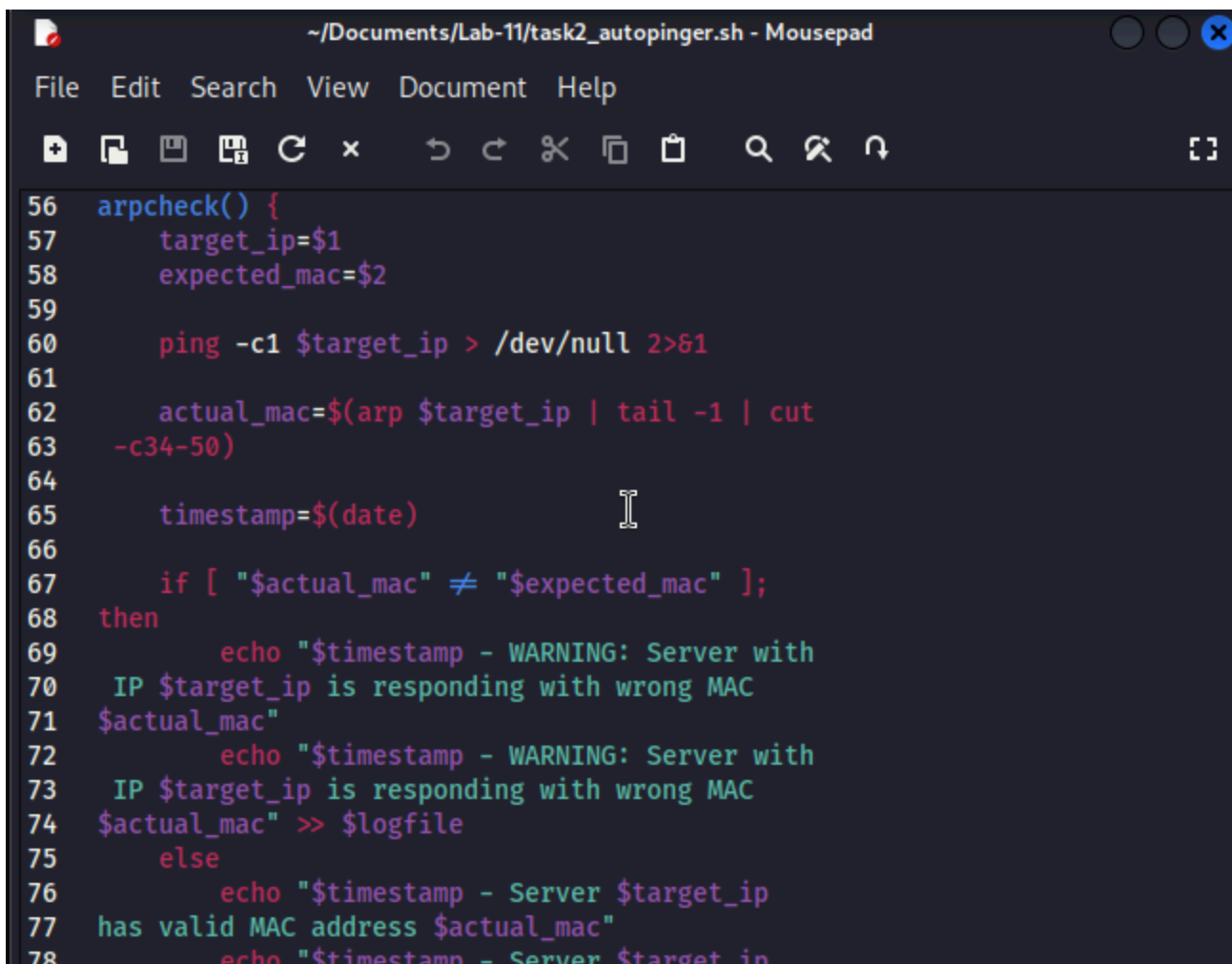
4. Why might logging and automation like this be valuable in detecting early signs of attack or system failure?

Automated logs capture changes in host availability over time, which manual checks would miss. A sudden spike in packet loss, a host going offline at an unusual hour, or repeated unreachability can all be early indicators of a DoS attack, a compromised host being shut down, or hardware failure — all detectable by reviewing the log history.

Task 5: Verifying MAC Address Integrity Using ARP

◆ Screenshots:

- ☐ arpcheck() function

A screenshot of a terminal window titled "~/Documents/Lab-11/task2_autopinger.sh - Mousepad". The window contains a shell script for the arpcheck() function. The script takes two arguments: target_ip and expected_mac. It pings the target IP and retrieves the actual MAC address from the ARP table. It then compares the actual MAC with the expected MAC. If they don't match, it prints a warning message with the timestamp, target IP, and actual MAC. If they match, it prints a confirmation message.

```
56 arpcheck() {
57     target_ip=$1
58     expected_mac=$2
59
60     ping -c1 $target_ip > /dev/null 2>&1
61
62     actual_mac=$(arp $target_ip | tail -1 | cut
63 -c34-50)
64
65     timestamp=$(date)
66
67     if [ "$actual_mac" ≠ "$expected_mac" ];
68 then
69         echo "$timestamp - WARNING: Server with
70 IP $target_ip is responding with wrong MAC
71 $actual_mac"
72         echo "$timestamp - WARNING: Server with
73 IP $target_ip is responding with wrong MAC
74 $actual_mac" >> $logfile
75     else
76         echo "$timestamp - Server $target_ip
77 has valid MAC address $actual_mac"
78         echo "$timestamp - Server $target ip
```

- ☐ Output for a correct and simulated incorrect MAC

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./task2_autopinger.sh
./task2_autopinger.sh: line 9: -2: command not found
./task2_autopinger.sh: line 11: -n1: command not found
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: line 15: errors: command not found
cut: you must specify a list of bytes, characters, or fields
Try 'cut --help' for more information.
./task2_autopinger.sh: line 22: -f3: command not found
Tue Apr 28 08:43:25 PM MDT 2026 - Server 192.168.10.6 is
responding with packet loss.
cut: you must specify a list of bytes, characters, or fields
Try 'cut --help' for more information.
./task2_autopinger.sh: line 64: -c34-50: command not found
Tue Apr 28 08:43:28 PM MDT 2026 - WARNING: Server with
IP 192.168.10.6 is responding with wrong MAC
```

- ☐ Log entries for both cases

```
(kevin@kali)-[~/Documents/Lab-11]
$ cat report.log
Tue Apr 28 08:28:06 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:33 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:28:38 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:42:00 PM MDT 2026 - Server 192.168.10.13 is
responding with packet loss.
Tue Apr 28 08:42:00 PM MDT 2026 - WARNING: Server with IP 192.168.10.13 is responding with wrong MAC ce:83:cf:61:c8:ba
Tue Apr 28 08:43:25 PM MDT 2026 - Server 192.168.10.6 is
responding with packet loss.
Tue Apr 28 08:43:28 PM MDT 2026 - WARNING: Server with
IP 192.168.10.6 is responding with wrong MAC
```

◆ Questions:

1. Explain what each part of the following command does: `arp $1 | tail -1 | cut -c34-50`

- `arp $1`: queries the ARP table for the given IP address and prints the entry including its MAC address.
- `tail -1`: takes only the last line of output, in case there are multiple entries or a header line.
- `cut -c34-50`: extracts characters 34 through 50, which is where the MAC address appears in the default `arp` output format.

2. Why is it important to validate the MAC address of a host? What attacks does this help detect?

MAC addresses are tied to physical network interfaces and should stay constant for a given device.

Validating them helps detect ARP spoofing, where an attacker sends forged ARP replies to associate their MAC with a legitimate IP, redirecting traffic through their machine for interception or a man-in-the-middle attack.

3. How could an attacker still bypass this check? (Consider spoofing or poisoning scenarios.)

An attacker could spoof their MAC address to match the expected value using tools like `macchanger`, making the check pass even though a different machine is responding. They could also poison the ARP

cache on the monitoring host itself before the check runs, so **arp** returns the attacker's MAC as if it were the legitimate one.

4. What are two ways to make this MAC verification more robust in production systems?

- Use a static ARP table (**arp -s**) to pin known IP-to-MAC mappings, preventing the cache from being poisoned by forged ARP replies.
- Cross-reference MAC addresses against a trusted DHCP lease database or network inventory tool (e.g., Nmap's OS/MAC fingerprinting) rather than relying solely on the local ARP cache.

Task 6: Port Availability Compliance Check

◆ Screenshots:

- ☐ `portcheckSimple()` function

```
portcheckSimple() {
    nmaplog=$(nmap $1 | grep $2)
    if [ "$nmaplog" != "" ]; then
        echo "$(date) WARNING: Port $2 is open
on $1"
        echo "$(date) WARNING: Port $2 is open
on $1" >> $logfile
    fi
}

pingtest 192.168.10.13
arpcheck 192.168.10.13| 00:0c:29:bc:e8:3b
portcheckSimple 192.168.10.13 139
portcheckSimple 192.168.10.13 100
```

- ☐ Output from checking port 139 and 100

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./task2_autopinger.sh
Tue Apr 28 08:48:58 PM MDT 2026 WARNING: Port 139 is open
on 192.168.10.13

(kevin@kali)-[~/Documents/Lab-11]
$
```

- ☐ Log entries showing open ports

```
17 192.168.10.13 is responding with wrong MAC
Tue Apr 28 08:48:35 PM MDT 2026 WARNING: Port 139 is open
on 192.168.10.13
Tue Apr 28 08:48:58 PM MDT 2026 WARNING: Port 139 is open
on 192.168.10.13
52
```

◆ Questions:

1. Why might port 139 be open on the MS-T machine but not on other modern systems?

Metasploitable 2 is a deliberately vulnerable legacy Linux system running Samba, which uses port 139 for NetBIOS Session Service. Modern systems have largely dropped NetBIOS in favor of SMB over port 445 directly, and most distributions disable Samba by default. Port 139 being open on MS-T reflects its intentionally outdated and insecure configuration.

2. What are the security implications of exposing legacy ports like 139 on a network?

Port 139 is associated with known vulnerabilities in older Samba and NetBIOS implementations, including remote code execution exploits. Exposing it allows attackers to enumerate network shares, brute-force credentials, or exploit unpatched services. It also reveals information about the OS and file-sharing configuration, giving attackers a clear attack surface.

3. How might this function be extended to support checking a list of ports or multiple IPs automatically?

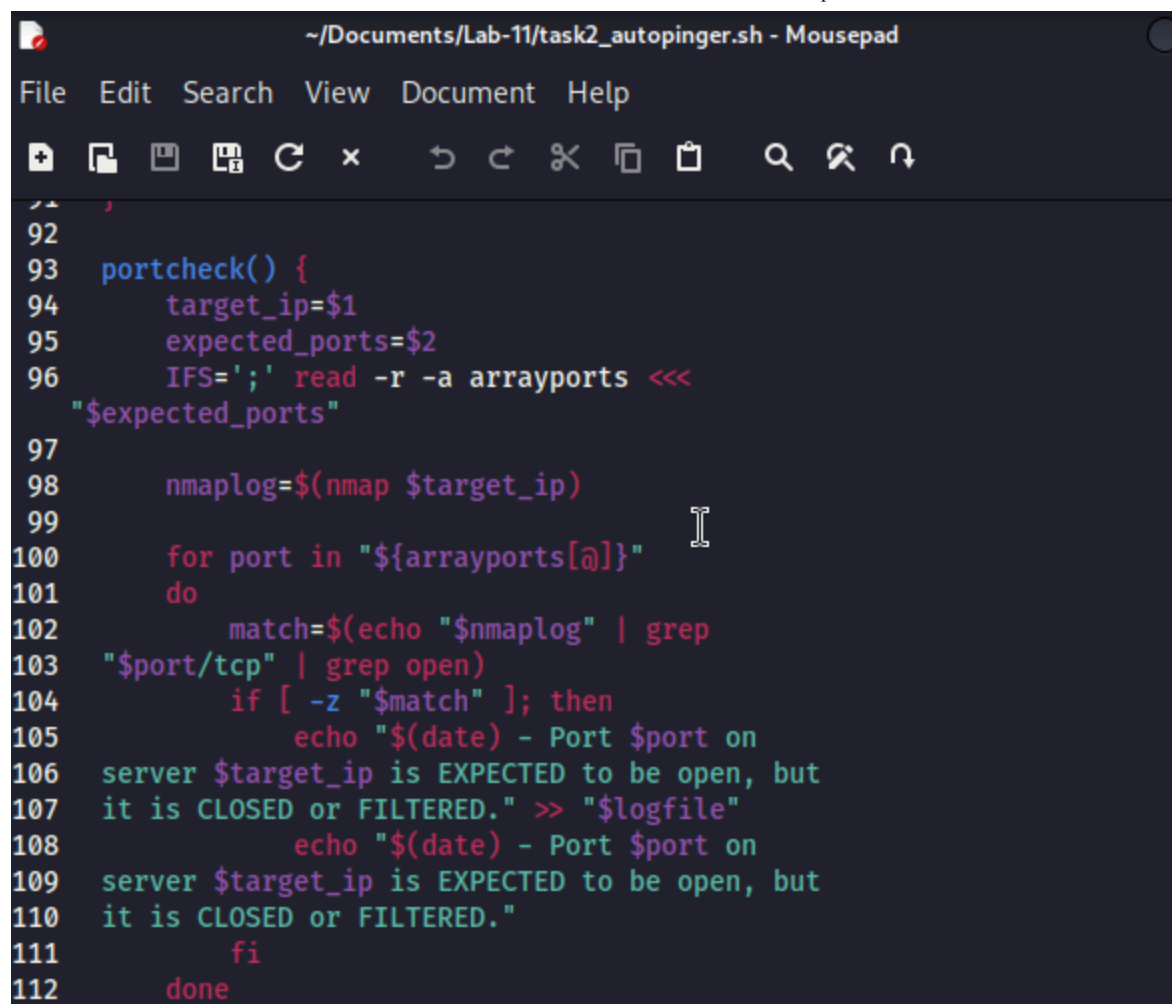
The function could be wrapped in a loop that iterates over an array of ports or reads IPs from a file, calling `portcheckSimple` for each combination. For example:

```
for ip in $(cat hosts.txt); do
    for port in 139 445 21 22 80; do
        portcheckSimple $ip $port
    done
done
```

Task 7: Advanced Port Compliance Verification

◆ Screenshots:

- ☐ `portcheck()` function



```
~/Documents/Lab-11/task2_autopinger.sh - Mousepad
File Edit Search View Document Help
[Icons]
92
93 portcheck() {
94     target_ip=$1
95     expected_ports=$2
96     IFS=';' read -r -a arrayports <<<
97     "$expected_ports"
98     nmaplog=$(nmap $target_ip)
99     for port in "${arrayports[@]}"; do
100         match=$(echo "$nmaplog" | grep
101         "$port/tcp" | grep open)
102         if [ -z "$match" ]; then
103             echo "$(date) - Port $port on
104             server $target_ip is EXPECTED to be open, but
105             it is CLOSED or FILTERED." >> "$logfile"
106             echo "$(date) - Port $port on
107             server $target_ip is EXPECTED to be open, but
108             it is CLOSED or FILTERED."
109         fi
110     done
111
112
```

- ☐ Output showing expected-closed and unexpected-open ports

```
(kevin@kali)-[~/Documents/Lab-11]
$ ./task2_autopinger.sh
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: line 104: 139/tcp: No such file or directory
Tue Apr 28 08:53:32 PM MDT 2026 - Port 139 on
server 192.168.10.13 is EXPECTED to be open, but
it is CLOSED or FILTERED.
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: line 104: 80/tcp: No such file or directory
Tue Apr 28 08:53:32 PM MDT 2026 - Port 80 on
server 192.168.10.13 is EXPECTED to be open, but
it is CLOSED or FILTERED.
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
./task2_autopinger.sh: line 104: 81/tcp: No such file or directory
Tue Apr 28 08:53:32 PM MDT 2026 - Port 81 on
server 192.168.10.13 is EXPECTED to be open, but
it is CLOSED or FILTERED.
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 21 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 22 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 23 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 25 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 53 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 80 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 111 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 139 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 445 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
```

- ☐ Log file reflecting both issues

```

(kevin@kali)-[~/Documents/Lab-11]
$ cat report.log
Tue Apr 28 08:28:06 PM MDT 2026 - Server 192.168.10.13 is
  responding with packet loss.
Tue Apr 28 08:28:33 PM MDT 2026 - Server 192.168.10.13 is
  responding with packet loss.
Tue Apr 28 08:28:38 PM MDT 2026 - Server 192.168.10.13 is
  responding with packet loss.
Tue Apr 28 08:42:00 PM MDT 2026 - Server 192.168.10.13 is
  responding with packet loss.
Tue Apr 28 08:42:00 PM MDT 2026 - WARNING: Server with IP 192.168.10.13 is respondi
Tue Apr 28 08:43:25 PM MDT 2026 - Server 192.168.10.6 is
  responding with packet loss.
Tue Apr 28 08:43:28 PM MDT 2026 - WARNING: Server with
  IP 192.168.10.6 is responding with wrong MAC

Tue Apr 28 08:48:35 PM MDT 2026 - Server 192.168.10.13 is
  responding with packet loss.
Tue Apr 28 08:48:35 PM MDT 2026 - WARNING: Server with
  IP 192.168.10.13 is responding with wrong MAC

Tue Apr 28 08:48:35 PM MDT 2026 WARNING: Port 139 is open
  on 192.168.10.13
Tue Apr 28 08:48:58 PM MDT 2026 WARNING: Port 139 is open
  on 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - Port 139 on
  server 192.168.10.13 is EXPECTED to be open, but
  it is CLOSED or FILTERED.
Tue Apr 28 08:53:32 PM MDT 2026 - Port 80 on
  server 192.168.10.13 is EXPECTED to be open, but
  it is CLOSED or FILTERED.
Tue Apr 28 08:53:32 PM MDT 2026 - Port 81 on
  server 192.168.10.13 is EXPECTED to be open, but
  it is CLOSED or FILTERED.
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
  port 21 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
  port 22 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
  port 23 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open

```

◆ Questions:

1. Why do we use **grep "\$port/tcp" | grep open** to confirm port status instead of just searching for the port number?

Searching for just the port number could match unrelated lines — for example, a port like **80** would also match **8080** or lines in the nmap header. Filtering for **\$port/tcp** ensures we match the exact port entry, and the second **grep open** confirms the port is actually open rather than closed or filtered, which nmap also reports in its output.

2. What types of misconfigurations or attacks might result in an unexpected port being open on a system?

A developer could leave a test service or debug server running (e.g., a Flask app on port 5000). An attacker who gained access may install a backdoor or reverse shell listening on an unusual port. Misconfigured software installations can also bind to default ports that weren't intended to be exposed externally.

3. How would you adapt this script to check multiple hosts (e.g., from a file)?

Read the file line by line and call `portcheck` for each host:

```
while IFS= read -r host; do
    portcheck "$host" "139;80;22"
done < hosts.txt
```

4. What should a system administrator do if unexpected ports are detected in production?

Immediately investigate what process is listening on the port using `ss -tlnp` or `lsof -i :<port>`. If the service is unauthorized, stop it and check for signs of compromise. Block the port at the firewall as a containment step, document the finding, and escalate according to the incident response policy. Review system logs to determine when and how the port was opened.

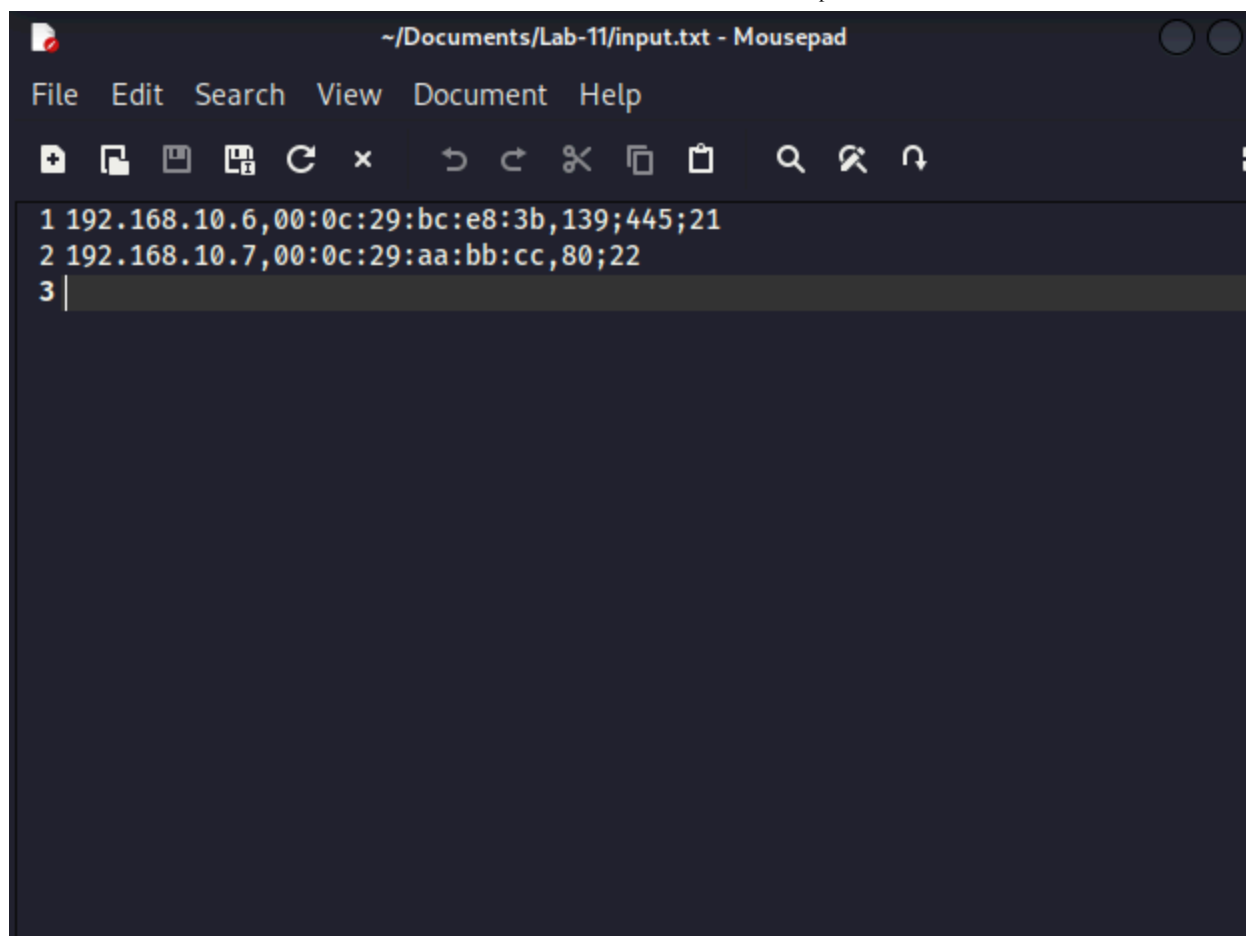
Task 8: Comprehensive Network Compliance Check from File Input

◆ Screenshots:

- ❑ `comprehensivecheck()` function

```
~/Documents/Lab-11/task2_autopinger.sh - Mousepad
File Edit Search View Document Help
[Icons] [Full Screen]

94 }
95
96 comprehensivecheck() {
97     input_file=$1
98
99     while IFS= read -r line; do
100         ip=$(echo "$line" | cut -d' ' -f1)
101         mac=$(echo "$line" | cut -d' ' -f2)
102         ports=$(echo "$line" | cut -d' ' -f3)
103
104         echo "$(date) - Starting compliance check for $ip" | tee -a
$logfile
105         pingtest "$ip"
106         arpcheck "$ip" "$mac"
107         portcheck "$ip" "$ports"
108         echo "$(date) - Compliance check complete for $ip" | tee -a
$logfile
109         echo "———"
110     done < "$input_file"
111 }
112
113 comprehensivecheck /home/kali/Documents/lab11/input.txt
114
```



- ☐ Terminal showing multiple host checks

```

$ mousepad task2_autopinger.sh
(kevin@kali) [~/Documents/Lab-11]
$ ./task2_autopinger.sh
tee: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Starting compliance check for 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21
ping: 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21: Name or service not known
Tue Apr 28 09:21:20 PM MDT 2026 - Server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is responding with packet loss.
./task2_autopinger.sh: line 28: /home/kevin/Documents/lab-11/report.log: No such file or directory
192.168.10.6,00:0c:29:bc:e8:3b,139;445;21: Unknown host
Tue Apr 28 09:21:20 PM MDT 2026 - WARNING: Server with IP 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is responding with wrong MAC
./task2_autopinger.sh: line 53: /home/kevin/Documents/lab-11/report.log: No such file or directory
Failed to resolve "192.168.10.6,00:0c:29:bc:e8:3b,139;445;21".
WARNING: No targets were specified, so 0 hosts scanned.
./task2_autopinger.sh: line 79: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Port 192.168.10.6,00:0c:29:bc:e8:3b,139 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but it is CLOSED or FILTERED.
./task2_autopinger.sh: line 79: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Port 445 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but it is CLOSED or FILTERED.
./task2_autopinger.sh: line 79: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Port 21 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but it is CLOSED or FILTERED.
tee: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Compliance check complete for 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21

tee: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Starting compliance check for 192.168.10.7,00:0c:29:aa:bb:cc,80;22
ping: 192.168.10.7,00:0c:29:aa:bb:cc,80;22: Name or service not known
Tue Apr 28 09:21:20 PM MDT 2026 - Server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is responding with packet loss.
./task2_autopinger.sh: line 28: /home/kevin/Documents/lab-11/report.log: No such file or directory
192.168.10.7,00:0c:29:aa:bb:cc,80;22: Unknown host
Tue Apr 28 09:21:20 PM MDT 2026 - WARNING: Server with IP 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is responding with wrong MAC
./task2_autopinger.sh: line 53: /home/kevin/Documents/lab-11/report.log: No such file or directory
Failed to resolve "192.168.10.7,00:0c:29:aa:bb:cc,80;22".
WARNING: No targets were specified, so 0 hosts scanned.
./task2_autopinger.sh: line 79: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Port 192.168.10.7,00:0c:29:aa:bb:cc,80 on server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is EXPECTED to be open, but it is CLOSED or FILTERED.
./task2_autopinger.sh: line 79: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Port 22 on server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is EXPECTED to be open, but it is CLOSED or FILTERED.
tee: /home/kevin/Documents/lab-11/report.log: No such file or directory
Tue Apr 28 09:21:20 PM MDT 2026 - Compliance check complete for 192.168.10.7,00:0c:29:aa:bb:cc,80;22

```

- ☐ Log file showing full compliance results

```

Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 514 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 1099 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 1524 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 2049 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 2121 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 3306 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 5432 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 5900 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 6000 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 6667 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 8009 found on server 192.168.10.13
Tue Apr 28 08:53:32 PM MDT 2026 - UNEXPECTED open
port 8180 found on server 192.168.10.13
Tue Apr 28 09:23:36 PM MDT 2026 - Starting compliance check for 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21
Tue Apr 28 09:23:36 PM MDT 2026 - Server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is responding with packet loss.
Tue Apr 28 09:23:36 PM MDT 2026 - WARNING: Server with IP 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is responding with wrong MAC
Tue Apr 28 09:23:36 PM MDT 2026 - Port 192.168.10.6,00:0c:29:bc:e8:3b,139 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but
it is CLOSED or FILTERED.
Tue Apr 28 09:23:36 PM MDT 2026 - Port 445 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but it is CLOSED or FILTERED.
Tue Apr 28 09:23:36 PM MDT 2026 - Port 21 on server 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21 is EXPECTED to be open, but it is CLOSED or FILTERED.
Tue Apr 28 09:23:36 PM MDT 2026 - Compliance check complete for 192.168.10.6,00:0c:29:bc:e8:3b,139;445;21
Tue Apr 28 09:23:36 PM MDT 2026 - Starting compliance check for 192.168.10.7,00:0c:29:aa:bb:cc,80;22
Tue Apr 28 09:23:36 PM MDT 2026 - Server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is responding with packet loss.
Tue Apr 28 09:23:36 PM MDT 2026 - WARNING: Server with IP 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is responding with wrong MAC
Tue Apr 28 09:23:36 PM MDT 2026 - Port 192.168.10.7,00:0c:29:aa:bb:cc,80 on server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is EXPECTED to be open, but it is
CLOSED or FILTERED.
Tue Apr 28 09:23:36 PM MDT 2026 - Port 22 on server 192.168.10.7,00:0c:29:aa:bb:cc,80;22 is EXPECTED to be open, but it is CLOSED or FILTERED.
Tue Apr 28 09:23:36 PM MDT 2026 - Compliance check complete for 192.168.10.7,00:0c:29:aa:bb:cc,80;22
kevin@kali:~/Documents/Lab-11$

```

◆ Questions:

1. Why is it useful to perform batch compliance checks from a file?

Running checks from a file lets you audit many hosts in one execution without modifying the script. It also makes the target list easy to update, version-control, and share across a team, which is essential in environments with dozens or hundreds of managed hosts.

2. What risks arise if the `input.txt` file is malformed? How would you add validation to the script?

If a line is missing a field, variables like `$mac` or `$ports` will be empty, causing `arpcheck` or `portcheck` to behave unpredictably or silently skip checks. Validation can be added by checking that each line has exactly three fields before processing:

```

if [ $(echo "$line" | awk '{print NF}') -ne 3 ]; then
    echo "Skipping malformed line: $line"
    continue
fi

```

3. How would you scale this script to support hundreds of servers in a real organization?

Run checks in parallel using background processes or `xargs -P` to avoid waiting for each host sequentially. Centralize logs to a shared location or a SIEM like Splunk, and break the input file into segments that can be distributed across multiple monitoring hosts. A proper orchestration tool like Ansible could also replace the script for large-scale environments.

4. Suggest one more compliance check you could add to this framework.

SSL/TLS certificate validation — using `openssl s_client` to check whether a host's certificate is expired or expiring soon. Expired certificates can cause service outages and are a common oversight in large organizations, making automated checking a valuable addition to any compliance framework.